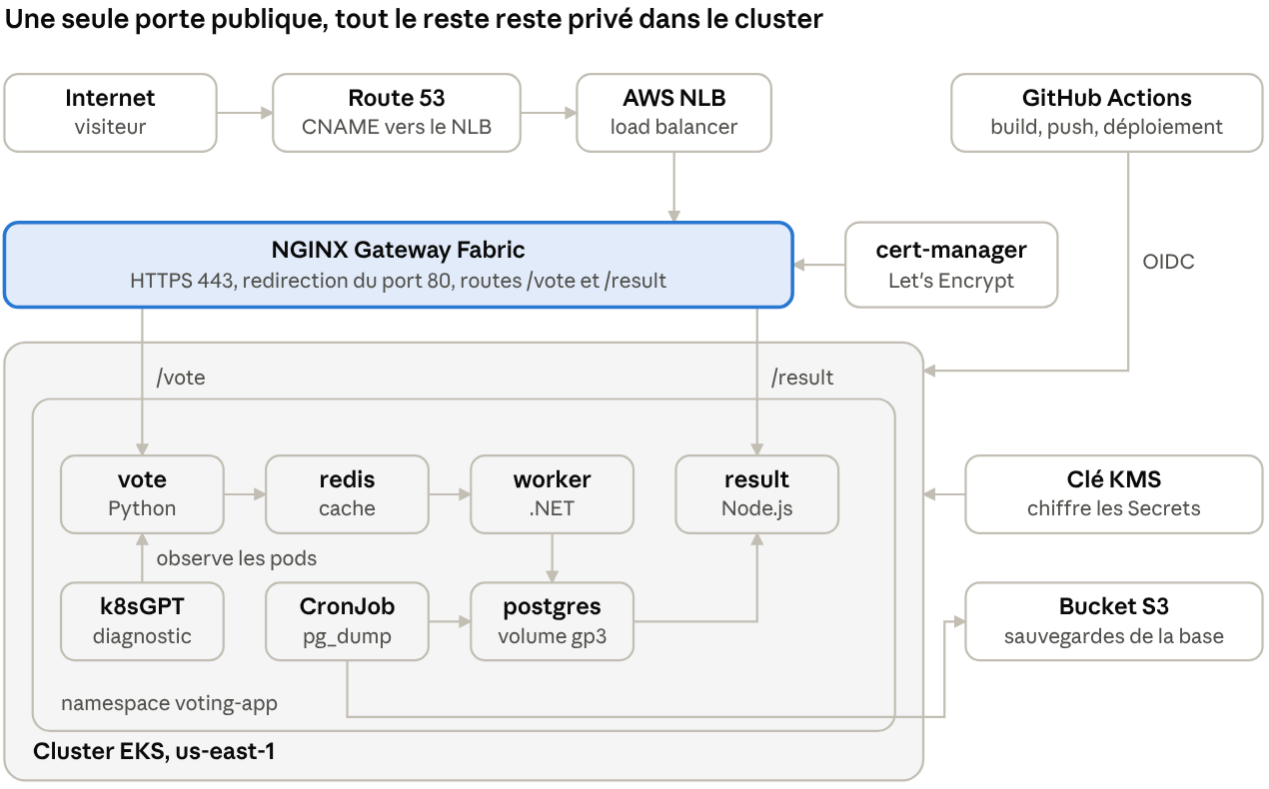
Guide pas à pas : déployer une application sur EKS avec CI/CD

Oct 27, 2026 · Dally NTOLO

Vue d'ensemble et prérequis

Ce guide déploie n'importe quelle application conteneurisée (interface, API, base de données) sur Amazon EKS, accessible en HTTPS et mise à jour automatiquement par GitHub Actions.

Chemin d'un visiteur : Internet, DNS Route 53, Network Load Balancer AWS, NGINX Gateway Fabric (seule porte publique), puis les services internes du cluster.



architecture du projet, une seule porte publique

Seule la Gateway est exposée. Les services applicatifs, la base et les sauvegardes restent à l'intérieur du cluster.

Outils à installer sur le poste :

- AWS CLI v2, configuré avec `aws configure`
- eksctl, kubectl, helm, git, docker

Il faut aussi : un compte AWS, un nom de domaine géré dans Route 53, un dépôt GitHub contenant le code et les manifestes Kubernetes, et des images Docker publiées dans un registre.

```
aws --version && eksctl version && kubectl version --client && helm version --short
aws sts get-caller-identity
```

Variables à adapter

On change ces valeurs une seule fois, toutes les commandes suivantes les réutilisent. Le reste du guide est identique quelle que soit l'application.

```
export AWS_REGION=us-east-1
export CLUSTER=mon-projet
export NS=mon-app
export DOMAIN=app.exemple.com
export GH_REPO=mon-compte/mon-depot
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export BUCKET=${CLUSTER}-pgbackup-${ACCOUNT_ID}
```

Variable	Rôle
CLUSTER	Nom du cluster EKS
NS	Namespace Kubernetes de l'application
DOMAIN	Nom de domaine public de l'application
GH_REPO	Dépôt GitHub autorisé à déployer

Étape 1 : créer le cluster EKS

Un fichier de configuration rend la création reproductible. Compter 15 à 20 minutes.

```
cat > cluster.yaml <<EOF
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: ${CLUSTER}
  region: ${AWS_REGION}
iam:
  withOIDC: true
managedNodeGroups:
  - name: ng-1
    instanceType: t3.medium
    desiredCapacity: 2
    minSize: 2
    maxSize: 3
EOF

eksctl create cluster -f cluster.yaml
kubectl get nodes
```

`withOIDC: true` est indispensable : il permet aux pods d'obtenir des droits AWS précis (étapes sauvegarde et volumes).

Volumes persistants (si l'application a une base de données). Installer le pilote EBS et une StorageClass gp3 :

```
eksctl create iamserviceaccount --name ebs-csi-controller-sa --namespace
kube-system \
  --cluster $CLUSTER --role-name ${CLUSTER}-ebs-csi --role-only \
  --attach-policy-arn arn:aws:iam::aws:policy/service-
role/AmazonEBSCSIDriverPolicy --approve

eksctl create addon --name aws-ebs-csi-driver --cluster $CLUSTER \
  --service-account-role-arn arn:aws:iam::${ACCOUNT_ID}:role/${CLUSTER}-ebs-
csi --force

kubectl apply -f - <<EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
volumeBindingMode: WaitForFirstConsumer
EOF
```

Étape 2 : chiffrer les Secrets avec une clé KMS

Par défaut, les Secrets Kubernetes sont stockés en base64 seulement. Une clé KMS ajoute un vrai chiffrement (envelope encryption).

```
KMS_ARN=$(aws kms create-key --description "${CLUSTER} secrets" \
  --query KeyMetadata.Arn --output text)
aws kms create-alias --alias-name alias/${CLUSTER}-secrets --target-key-id
$KMS_ARN

eksctl utils enable-secrets-encryption --cluster $CLUSTER \
  --key-arn $KMS_ARN --region $AWS_REGION
```

L'opération prend quelques minutes et ne coupe pas l'application.

Étape 3 : déployer l'application

On crée le namespace, les Secrets, puis les manifestes dans l'ordre : base de données, cache, services applicatifs, interface. Tous les Services restent en `ClusterIP` : rien n'est exposé à Internet à ce stade.

```
kubectl create namespace $NS

# Le mot de passe ne va jamais dans Git
kubectl -n $NS create secret generic db-credentials \
  --from-literal=POSTGRES_USER=app \
  --from-literal=POSTGRES_PASSWORD='<choisir-un-mot-de-passe>'

kubectl apply -n $NS -f k8s/
kubectl -n $NS rollout status deploy/<nom-du-deployment>
kubectl -n $NS get pods,svc,pvc
```

Critère de réussite : tous les pods sont `Running`, le PVC de la base est `Bound`.

Étape 4 : Gateway API et NGINX Gateway Fabric

La Gateway API est le successeur officiel de l'Ingress : elle sépare la porte d'entrée (Gateway) des règles de routage (HTTPRoute), sans annotations propres à un produit. NGINX Gateway Fabric en est l'implémentation basée sur NGINX.

Vérifier dans la documentation de NGINX Gateway Fabric la version de la Gateway API compatible avant d'installer.

```
kubectl apply -f https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.2.1/standard-install.yaml

helm install ngf oci://ghcr.io/nginx/charts/nginx-gateway-fabric \
  --create-namespace -n nginx-gateway

kubectl get gatewayclass
```

Créer ensuite la Gateway et les routes (`gateway.yaml`) :

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: gw
  namespace: mon-app
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      port: 80
      protocol: HTTP
    - name: https
      port: 443
      protocol: HTTPS
      hostname: app.exemple.com
      tls:
        mode: Terminate
        certificateRefs:
          - name: app-tls
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: routes
  namespace: mon-app
spec:
  parentRefs:
    - name: gw
      sectionName: https
  hostnames: ["app.exemple.com"]
  rules:
    - matches: [{path: {type: PathPrefix, value: /}}]
      backendRefs: [{name: front, port: 80}]
    - matches: [{path: {type: PathPrefix, value: /api}}]
      backendRefs: [{name: api, port: 8080}]
---
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: http-redirect
  namespace: mon-app
spec:
  parentRefs:
    - name: gw
      sectionName: http
  rules:
    - filters:
        - type: RequestRedirect
          requestRedirect: {scheme: https, statusCode: 301}
```

Adapter le domaine, les chemins, les noms de Services et les ports. Appliquer avec `kubectl apply -f gateway.yaml`. AWS crée alors automatiquement un load balancer :

```
kubectl get svc -A | grep LoadBalancer
```

Étape 5 : DNS et certificat HTTPS

1. Pointer le domaine vers le load balancer (enregistrement CNAME dans Route 53) :

```
LB=$(kubectl get svc -A -o jsonpath='{.items[?
(@.spec.type=="LoadBalancer")].status.loadBalancer.ingress[0].hostname}')
ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name <domaine-racine> \
  --query 'HostedZones[0].Id' --output text)

aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-
batch "{
  \"Changes\": [{\"Action\": \"UPSERT\", \"ResourceRecordSet\": {
    \"Name\": \"${DOMAIN}\", \"Type\": \"CNAME\", \"TTL\": 60,
    \"ResourceRecords\": [{\"Value\": \"${LB}\"}]}}]}"
```

2. Installer cert-manager :

```
helm repo add jetstack https://charts.jetstack.io && helm repo update
helm install cert-manager jetstack/cert-manager -n cert-manager \
  --create-namespace --set crds.enabled=true
```

3. Demander le certificat Let's Encrypt (validation HTTP-01 via la Gateway) :

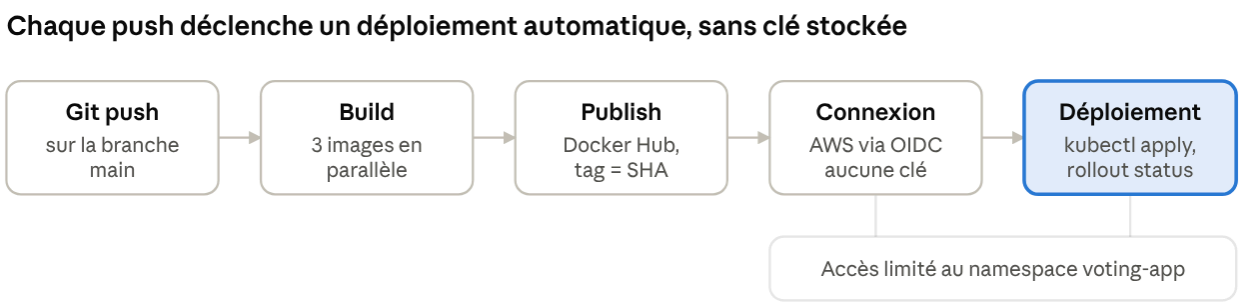
```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: votre.email@exemple.com
    privateKeySecretRef:
      name: letsencrypt-key
    solvers:
      - http01:
          gatewayHTTPRoute:
            parentRefs:
              - name: gw
                namespace: mon-app
                kind: Gateway
                sectionName: http
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: app-tls
  namespace: mon-app
spec:
  secretName: app-tls
  dnsNames: ["app.exemple.com"]
  issuerRef:
    name: letsencrypt
    kind: ClusterIssuer
```

```
kubectl apply -f issuer.yaml
kubectl -n $NS get certificate -w     # attendre READY = True
curl -I https://$DOMAIN
```

Astuce : tant que le DNS n'est pas propagé, la validation échoue. Attendre quelques minutes avant de relancer.

Étape 6 : CI/CD avec GitHub Actions et OIDC

GitHub Actions s'authentifie auprès d'AWS par OIDC : aucune clé d'accès n'est stockée dans GitHub. Le rôle obtenu est limité à un seul namespace.



pipeline GitHub Actions, 5 étapes

De la gauche vers la droite : du push au déploiement, sans clé AWS stockée dans GitHub.

1. Fournisseur OIDC GitHub (une seule fois par compte AWS ; s'il existe déjà, ne pas le recréer ni le supprimer) :

```
aws iam list-open-id-connect-providers
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com
```

2. Rôle IAM que GitHub peut endosser :

```
cat > trust.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated": "arn:aws:iam::${ACCOUNT_ID}:oidc-
provider/token.actions.githubusercontent.com"},
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {"token.actions.githubusercontent.com:aud":
"sts.amazonaws.com"},
      "StringLike": {"token.actions.githubusercontent.com:sub":
"repo:${GH_REPO}:ref:refs/heads/main"}
    }
  }]
}
EOF

aws iam create-role --role-name ${CLUSTER}-github-deploy \
  --assume-role-policy-document file://trust.json
```

```
aws iam put-role-policy --role-name ${CLUSTER}-github-deploy --policy-name
eks-describe \
  --policy-document '{"Version":"2012-10-17","Statement":
[{"Effect":"Allow","Action":"eks:DescribeCluster","Resource":"*"}]}'
```

3. Donner au rôle l'accès au cluster, limité au namespace :

```
ROLE_ARN=arn:aws:iam::${ACCOUNT_ID}:role/${CLUSTER}-github-deploy

aws eks create-access-entry --cluster-name $CLUSTER --principal-arn $ROLE_ARN
aws eks associate-access-policy --cluster-name $CLUSTER --principal-arn
$ROLE_ARN \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSEditPolicy \
  --access-scope type=namespace,namespaces=$NS
```

Point d'attention : l'ARN du rôle dans l'access entry, la condition `sub` (dépôt et branche) et le rôle renseigné dans le workflow doivent correspondre exactement. Un seul écart et le déploiement est refusé.

4. Workflow (`.github/workflows/deploy.yml`) :

```
name: deploy
on:
  push:
    branches: [main]
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<ACCOUNT_ID>:role/<CLUSTER>-github-
deploy
          aws-region: us-east-1
      - run: aws eks update-kubeconfig --name <CLUSTER> --region us-east-1
      - run: kubectl apply -n <NS> -f k8s/
      - run: kubectl -n <NS> rollout status deploy/<nom-du-deployment>
```

Pour construire et publier les images avant le déploiement, ajouter une étape `docker/login-action` puis `docker/build-push-action` avec `tags: <registre>/<image>:${{ github.sha }}`.

Étape 7 : analyse de qualité du code avec SonarQube

SonarQube analyse le code à chaque push (bugs, failles, code dupliqué) et peut bloquer le déploiement si le seuil de qualité (Quality Gate) n'est pas atteint. Étape optionnelle, à placer avant le déploiement. Deux options :

Option	Quand la choisir
SonarQube Cloud (hébergé)	Le plus simple : rien à installer, adapté à un dépôt public
SonarQube Community dans le cluster	Les sources doivent rester chez soi

Option A : SonarQube Cloud. Créer un compte avec GitHub, importer le dépôt, générer un jeton, puis le stocker dans GitHub :

```
gh secret set SONAR_TOKEN --repo $GH_REPO
```

Option B : SonarQube dans le cluster.

```
helm repo add sonarqube https://SonarSource.github.io/helm-chart-sonarqube &&
helm repo update
helm install sonarqube sonarqube/sonarqube -n sonarqube --create-namespace \
  --set community.enabled=true \
  --set monitoringPasscode='<mot-de-passe>'
kubectl -n sonarqube get pods -w
kubectl -n sonarqube port-forward svc/sonarqube-sonarqube 9000:9000
```

Ouvrir http://localhost:9000, changer le mot de passe administrateur, créer un projet et un jeton. Pour que GitHub Actions joigne le serveur, il doit être accessible depuis Internet (par exemple via une route sur la Gateway) : stocker alors son adresse dans le secret `SONAR_HOST_URL`. Vérifier les valeurs du chart dans sa documentation, elles évoluent selon les versions.

Fichier `sonar-project.properties` à la racine du dépôt :

```
sonar.projectKey=mon-projet
sonar.organization=mon-organisation   # Option A uniquement
sonar.sources=.
```

Job à ajouter dans le workflow, avant le déploiement :

```
  sonar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: SonarSource/sonarqube-scan-action@v5
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}   # Option B
uniquement
      - uses: SonarSource/sonarqube-quality-gate-action@v1
        timeout-minutes: 5
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}

  deploy:
    needs: sonar      # le déploiement n'a lieu que si l'analyse passe
```

Critère de réussite : le job `sonar` est vert et le résultat apparaît dans le tableau de bord SonarQube.

Étape 8 : sécurité et résilience

NetworkPolicies. Elles limitent qui peut parler à qui. Sur EKS, il faut d'abord activer leur application dans le plugin réseau :

```
aws eks update-addon --cluster-name $CLUSTER --addon-name vpc-cni \
  --configuration-values '{"enableNetworkPolicy":"true"}'
```

Puis tout interdire par défaut et autoriser les flux utiles :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: mon-app
spec:
  podSelector: {}
  policyTypes: [Ingress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
```

```
  name: allow-api-to-db
  namespace: mon-app
spec:
  podSelector:
    matchLabels: {app: postgres}
  ingress:
    - from:
        - podSelector:
            matchLabels: {app: api}
      ports:
        - port: 5432
```

Penser à autoriser aussi le trafic venant de la Gateway vers les services exposés, sinon l'application devient inaccessible.

Mise à l'échelle automatique (HPA). Les Deployments doivent déclarer des `resources.requests`.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
kubectl -n $NS autoscale deployment <nom> --cpu-percent=70 --min=2 --max=5
kubectl -n $NS get hpa
```

Diagnostic assisté avec k8sGPT (optionnel). L'opérateur analyse les pods en continu :

```
helm repo add k8sgpt https://charts.k8sgpt.ai && helm repo update
helm install k8sgpt-operator k8sgpt/k8sgpt-operator -n k8sgpt-operator-system
--create-namespace
kubectl get results -A
```

Étape 9 : sauvegarde Postgres vers S3

Un CronJob exporte la base chaque jour avec `pg_dump` et dépose le fichier dans un bucket S3 chiffré, conservé 7 jours.

1. Bucket et règle de conservation :

```
aws s3api create-bucket --bucket $BUCKET --region $AWS_REGION
aws s3api put-public-access-block --bucket $BUCKET --public-access-block-
configuration \

BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPub
licBuckets=true
aws s3api put-bucket-encryption --bucket $BUCKET --server-side-encryption-
configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":
{"SSEAlgorithm":"aws:kms"}}]}'
aws s3api put-bucket-lifecycle-configuration --bucket $BUCKET --lifecycle-
configuration \
  '{"Rules":[{"ID":"expire-7d","Status":"Enabled","Filter":{},"Expiration":
{"Days":7}}]}'
```

(En `us-east-1`, la commande `create-bucket` n'accepte pas l'option `--create-bucket-configuration`. Dans une autre région, l'ajouter.)

2. Droits d'écriture seule, rattachés à un compte de service :

```
POLICY_ARN=$(aws iam create-policy --policy-name ${CLUSTER}-pgbackup-write \
  --policy-document "{\"Version\":\"2012-10-17\",\"Statement\":
[{\"Effect\":\"Allow\",\"Action\":\"s3:PutObject\",\"Resource\":\"arn:aws:s3:
::${BUCKET}/*\"}]}" \
  --query Policy.Arn --output text)

eksctl create iamserviceaccount --name pgbackup --namespace $NS --cluster
$CLUSTER \
  --attach-policy-arn $POLICY_ARN --approve
```

Si le bucket est chiffré avec une clé KMS gérée par le client, ajouter `kms:GenerateDataKey` sur cette clé dans la policy.

3. CronJob (`backup.yaml`) :

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: pg-backup
  namespace: mon-app
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
```

```
          serviceAccountName: pgbackup
          restartPolicy: OnFailure
          volumes:
            - name: dump
              emptyDir: {}
          initContainers:
            - name: dump
              image: postgres:16
              envFrom:
                - secretRef: {name: db-credentials}
              command: ["sh", "-c", "PGPASSWORD=$POSTGRES_PASSWORD pg_dump -h
postgres -U $POSTGRES_USER -Fc <nom-base> > /dump/backup.dump"]
              volumeMounts:
                - {name: dump, mountPath: /dump}
          containers:
            - name: upload
              image: amazon/aws-cli
              command: ["sh", "-c", "aws s3 cp /dump/backup.dump
s3://<BUCKET>/backup-$(date +%F).dump"]
              volumeMounts:
                - {name: dump, mountPath: /dump}
```

Remplacer `<BUCKET>`, le nom du Service de la base (`postgres`) et `<nom-base>`.

4. Test immédiat, sans attendre la nuit :

```
kubectl apply -f backup.yaml
kubectl -n $NS create job --from=cronjob/pg-backup test-backup
kubectl -n $NS logs job/test-backup -c upload
aws s3 ls s3://$BUCKET/
```

Étape 10 : vérifications et nettoyage

Vérifier que tout fonctionne :

```
kubectl -n $NS get pods,svc,pvc,hpa,cronjob
kubectl -n $NS get gateway,httproute,certificate
curl -I http://$DOMAIN     # doit répondre 301 vers HTTPS
curl -I https://$DOMAIN    # doit répondre 200
```

Puis faire un `git push` sur `main` et vérifier que le workflow passe au vert.

Nettoyage (pour ne plus payer). Respecter cet ordre : un load balancer ou un volume oublié continue de coûter, et supprimer le cluster d'abord peut les laisser orphelins.

```
# 1. DNS : supprimer l'enregistrement CNAME du domaine (console Route 53)
# 2. Load balancer et volumes
kubectl delete -n $NS -f gateway.yaml
kubectl -n $NS delete pvc --all
# 3. Cluster
eksctl delete cluster --name $CLUSTER --region $AWS_REGION --wait
# 4. IAM
aws iam delete-role-policy --role-name ${CLUSTER}-github-deploy --policy-name
eks-describe
aws iam delete-role --role-name ${CLUSTER}-github-deploy
aws iam delete-policy --policy-arn $POLICY_ARN
# 5. Sauvegardes
aws s3 rm s3://$BUCKET --recursive && aws s3 rb s3://$BUCKET
# 6. Clé KMS (suppression différée, 7 jours minimum)
aws kms delete-alias --alias-name alias/${CLUSTER}-secrets
aws kms schedule-key-deletion --key-id $KMS_ARN --pending-window-in-days 7
```

À ne jamais supprimer : le fournisseur OIDC GitHub et la zone Route 53 du domaine, car ils peuvent servir à d'autres projets.

Annexe A : vider le cluster sans le supprimer

Utile pour repartir de zéro sur le même cluster, ou pour arrêter les coûts d'une application en gardant l'infrastructure. Respecter l'ordre : la Gateway d'abord, pour que AWS supprime le load balancer.

```
# 1. Application, routes et load balancer
kubectl -n $NS delete gateway,httproute,certificate --all
kubectl delete clusterissuer letsencrypt
kubectl -n $NS delete cronjob,job --all
kubectl delete namespace $NS          # supprime aussi les PVC, donc les
volumes EBS

# 2. Composants installés avec Helm
helm uninstall k8sgpt-operator -n k8sgpt-operator-system
helm uninstall sonarqube -n sonarqube
helm uninstall ngf -n nginx-gateway
helm uninstall cert-manager -n cert-manager
kubectl delete namespace k8sgpt-operator-system sonarqube nginx-gateway cert-
manager

# 3. Définitions de la Gateway API
kubectl delete -f https://github.com/kubernetes-sigs/gateway-
api/releases/download/v1.2.1/standard-install.yaml
```

```
# 4. Vérifier qu'il ne reste rien qui coûte
kubectl get svc -A | grep LoadBalancer     # doit être vide
kubectl get pv                              # doit être vide
kubectl get ns
```

Si seuls les nœuds doivent s'arrêter (le cluster lui-même reste facturé environ 0,10 dollar par heure) :

```
eksctl scale nodegroup --cluster $CLUSTER --name ng-1 --nodes 0 --nodes-min 0
```

Pour tout supprimer, y compris le cluster, suivre le nettoyage complet de l'étape 10.

Annexe B : parties non réalisées dans le projet

Ces trois points restaient ouverts à la fin du projet. Ils sont documentés ici pour être repris.

1. Donner un moteur d'IA à k8sGPT. Sans moteur, k8sGPT détecte les problèmes mais ne les explique pas en langage clair. Stocker la clé dans un Secret, jamais dans Git ni dans une conversation :

```
kubectl -n k8sgpt-operator-system create secret generic k8sgpt-ai-secret \
  --from-literal=api-key='<clé-du-fournisseur>'
```

```
apiVersion: core.k8sgpt.ai/v1alpha1
kind: K8sGPT
metadata:
  name: k8sgpt
  namespace: k8sgpt-operator-system
spec:
  ai:
    enabled: true
    backend: openai
    model: gpt-4o-mini
    secret:
      name: k8sgpt-ai-secret
      key: api-key
  repository: ghcr.io/k8sgpt-ai/k8sgpt
  version: <version-compatible-de-l-opérateur>
```

Puis `kubectl apply -f k8sgpt.yaml` et `kubectl get results -A`. Vérifier le schéma exact dans la documentation de l'opérateur pour la version installée.

2. Empêcher le double vote (ou tout abus d'une page publique). Deux niveaux complémentaires :

- côté application : mémoriser un identifiant de votant (cookie signé ou identifiant de session) et refuser un second vote ;
- côté entrée : limiter le nombre de requêtes par adresse IP au niveau de la Gateway (fonction de limitation de débit de NGINX Gateway Fabric, à vérifier dans sa documentation selon la version).

3. Nettoyage des fichiers de débogage. Retirer du dépôt le workflow `oidc-debug.yml` utilisé pour diagnostiquer l'identité OIDC, et supprimer les namespaces de démonstration :

```
git rm .github/workflows/oidc-debug.yml && git commit -m "Remove OIDC debug workflow" && git push
kubectl delete namespace demo-panne
```

Annexe C : variantes avec les technologies non utilisées

Le guide utilise eksctl pour l'infrastructure et `kubectl apply -f` pour l'application. Voici ce qui change avec les autres outils.

Outil	Remplace	Apport
Terraform	eksctl et les commandes `aws`	Infrastructure décrite en fichiers, `plan` avant `apply`, `destroy` complet
Helm	`kubectl apply -f` pour l'application	Valeurs paramétrables, historique, `rollback`
Kustomize	`kubectl apply -f` pour l'application	Manifestes de base et variantes par environnement, sans modèles

Les manifestes Kubernetes, la Gateway API, cert-manager et le CronJob de sauvegarde restent identiques.

Terraform à la place d'eksctl. Squelette avec le module communautaire (adapter les versions) :

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name    = var.cluster
  cluster_version = "<version-kubernetes-supportée>"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
  enable_irsa     = true     # équivaut à withOIDC: true

  eks_managed_node_groups = {
    ng-1 = { instance_types = ["t3.medium"], min_size = 2, max_size = 3,
desired_size = 2 }
  }

  cluster_addons = { aws-ebs-csi-driver = { most_recent = true } }
}
```

Le VPC peut venir du module `terraform-aws-modules/vpc/aws`. La clé KMS, le bucket S3, le rôle GitHub et les policies deviennent des ressources `aws_kms_key`, `aws_s3_bucket`, `aws_iam_role` et `aws_iam_policy`. Ensuite :

```
terraform init && terraform plan && terraform apply
aws eks update-kubeconfig --name $CLUSTER --region $AWS_REGION
```

La suppression se réduit à `terraform destroy`, après avoir supprimé la Gateway et les PVC (sinon le load balancer et les volumes créés par Kubernetes bloquent la destruction du réseau).

Helm pour l'application. Les manifestes deviennent un chart avec un fichier de valeurs :

```
helm create mon-app
# Adapter templates/ et values.yaml (domaine, images, réplicas)
helm upgrade --install mon-app ./mon-app -n $NS --create-namespace -f values-
prod.yaml
helm history mon-app -n $NS
helm rollback mon-app 1 -n $NS
```

Kustomize pour l'application. Arborescence `base/` (manifestes communs) et `overlays/prod/` (différences) :

```
kubectl kustomize overlays/prod       # voir le résultat sans rien appliquer
kubectl apply -k overlays/prod

# Dans la CI : changer le tag d'une image
cd overlays/prod && kustomize edit set image front=registre/front:$GITHUB_SHA
```

Dans le workflow GitHub Actions, il suffit de remplacer l'étape `kubectl apply -n <NS> -f k8s/` par `kubectl apply -k overlays/prod` ou par `helm upgrade --install ...`.

Autres alternatives à connaître :

Choix du guide	Alternative	Ce qui change
Images sur Docker Hub	Amazon ECR	Registre privé dans AWS, accès géré par IAM
Gateway API et NGINX Gateway Fabric	Ingress avec AWS Load Balancer Controller	Un Application Load Balancer par Ingress, certificat via AWS Certificate Manager au lieu de cert-manager
Déploiement par kubectl dans GitHub Actions	Argo CD ou Flux (GitOps)	Le cluster se synchronise seul depuis Git, la CI ne touche plus au cluster
Nœuds EC2 gérés	AWS Fargate	Plus de nœuds à gérer, facturation par pod
Postgres dans le cluster	Amazon RDS	Sauvegardes et mises à jour gérées par AWS, plus de CronJob à maintenir

Exemple pour passer à ECR :

```
aws ecr create-repository --repository-name vote --region $AWS_REGION
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS
--password-stdin ${ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com
docker tag vote:1.0
${ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/vote:1.0
docker push ${ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com/vote:1.0
```